

Evolução do Simulador de Recarga em Python

Estruturas de Dados e Algoritmos — Sprint 3

ChargeGrid Intelligence · EV Challenge 2026 · FIAP + GoodWe

Disciplina: Data Structures and Algorithms

Hardware de referência: carregador AC GoodWe GW11K-HCA-20 (11 kW)

Sumário executivo

A entrega desta Sprint **não é um programa separado**. Os algoritmos de busca e de ordenação pedidos pelo enunciado foram implementados dentro do ChargeGrid Intelligence, o sistema de gestão de eletropostos que este grupo vem construindo, e **rodam em produção**:

- a **busca sequencial** é o que responde `SessionManager.get_session`, chamado a cada consulta de sessão pela interface web;
- o **insertion sort** é o que ordena a fila de restauração de potência em `PowerManager.rebalance`, executado toda vez que um conector é liberado.

O enunciado pede a "evolução do simulador" e exige, no item de análise de complexidade, que "a análise deverá utilizar o código efetivamente desenvolvido". Analisar algoritmos que o sistema usa de verdade é a forma mais literal de atender isso.

O mesmo sistema tem dois pontos de entrada:

Comando	O que abre
<code>python app.py</code>	interface web completa (Flask)
<code>python menu.py</code>	menu de terminal desta Sprint, sobre a mesma coleção
<code>python menu.py --autoteste</code>	bateria de asserções, sem abrir o menu

`menu.py` **não importa Flask**: roda com Python puro, sem nenhuma dependência instalada.

1. Onde encontrar cada item do enunciado

Todas as linhas abaixo foram conferidas por script contra os arquivos desta entrega (ver §9).

#	Exigência	Onde está
1	Estrutura para representar uma sessão (classe)	<code>models.py:68</code> — <code>ChargingSession</code> , <code>dataclass</code> de 17 campos
2	Lista para registrar múltiplas sessões	<code>session_manager.py:79</code> — <code>_sessions: List[ChargingSession]</code> ; <code>append</code> em <code>session_manager.py:172</code> ; <code>len()</code> em <code>session_manager.py:483</code> e <code>menu.py:386</code>
3	Menu principal com <code>while True</code>	<code>menu.py:669</code> — função <code>main()</code>
4	Busca implementada manualmente	<code>algoritmos.py:69</code> (sequencial) e <code>algoritmos.py:102</code> (binária); usadas em <code>session_manager.py:438</code> e <code>menu.py:400</code>
5	Ordenação implementada manualmente, critério à escolha	<code>algoritmos.py:151</code> (bubble) e <code>algoritmos.py:188</code> (insertion); critérios em <code>algoritmos.py:237</code> ; usadas em <code>power_manager.py:396</code> e <code>menu.py:468</code>

#	Exigência	Onde está
6	Estatísticas (mínimo 6 valores)	<code>algoritmos.py:249</code> — 6 valores exigidos + 4 de apoio; exibidas em <code>menu.py:519</code>
7	Funções organizando as operações	<code>algoritmos.py</code> (8 funções) e <code>menu.py</code> (18 funções), além dos 16 módulos do sistema
8	Validações de entrada	<code>menu.py:124</code> (<code>ler_numero</code>), <code>menu.py:96</code> (<code>ler_texto</code>), <code>menu.py:186</code> (<code>ler_opcao</code>), <code>algoritmos.py:327</code> (ID duplicado), <code>menu.py:669</code> (rede de segurança do laço)
9	Análise Big-O de dois algoritmos do projeto	§6 deste documento — os quatro são analisados
10	Comparação com significado prático	§7 deste documento, com números medidos por <code>menu.py:552</code> (opção 6 do menu)

2. Estrutura de dados utilizada

2.1 A sessão é uma classe

O enunciado pede "preferencialmente uma classe". `ChargingSession` é uma `@dataclass` de 17 campos, que já existia no sistema desde a Sprint 2 e que ganhou nesta Sprint o campo `numero`:

`models.py`, linhas 119–126 — o ID sequencial acrescentado nesta Sprint

```
# `numero` é o ID sequencial que o operador digita no menu de terminal.
# Existe ao lado do `session_id` porque os dois resolvem problemas
# diferentes: o UUID garante unicidade global (é o que vai para o banco,
# para o QR Code e para o recibo), enquanto um inteiro curto e crescente
# é o que um humano consegue digitar e o que a busca binária precisa como
# chave ordenável. Default 0 = "ainda não numerada"; quem numera é
# SessionManager (via algoritmos.proximo_numero) ou db.carregar_sesoes.
numero: int = 0
```

Por que dois identificadores. O `session_id` (`CGI-1480B94E`) é um UUID: garante unicidade global e é o que vai para o banco, para o QR Code e para o recibo. Mas ninguém digita isso num terminal, e a busca binária precisa de uma chave ordenável e curta. O `numero` resolve os dois problemas sem quebrar nada do que já existia — o campo tem valor padrão, então todo código anterior continua construindo sessões do mesmo jeito.

Os cinco atributos opcionais sugeridos pelo enunciado **já existiam** no sistema, porque são informação de negócio e não enfeite da prova:

Atributo sugerido	Campo em <code>ChargingSession</code>	Para que o sistema usa
veículo	<code>vehicle_id</code>	identificação no recibo e no painel do posto
carregador	<code>charger_id</code>	ocupação de conector e roteamento Modbus
horário	<code>start_time</code> / <code>end_time</code>	tarifação por faixa horária
tarifa	<code>tariff_kwh</code>	eixo de preço da <code>PricingEngine</code>
status	<code>status</code>	ciclo de vida, espelha o registrador Modbus 10017

2.2 O tempo é derivado, não é um campo

Esta é a única divergência consciente em relação ao exemplo do enunciado, que trata `tempo` como atributo do construtor. No ChargeGrid a duração é uma propriedade calculada:

`models.py`, linhas 169–172

```
@property
def duration_minutes(self) -> float:
    """Duração da sessão em minutos."""
    return self.duration_seconds / 60.0
```

Por quê. Energia é `potência × tempo`, e o motor de energia acumula kWh a partir do intervalo real entre dois carimbos de tempo. Se a duração fosse um campo independente, ela poderia divergir do intervalo que gerou a energia — e um sistema de faturamento não pode ter dois valores de duração que discordam. Com a propriedade derivada existe **uma única fonte de verdade**.

O custo dessa escolha aparece no cadastro manual: quando o operador digita "95 minutos", o menu converte esse número em `end_time`. A conversão está em `menu.py:354` e é conferida no autoteste ("tempo digitado vira `end_time` sem perda").

2.3 A coleção de sessões é uma lista

`session_manager.py`, linha 79 — a estrutura que o enunciado pede

```
self._sessions: List[ChargingSession] = []
```

Na Sprint 2 esta estrutura era um `Dict[str, ChargingSession]` indexado por `session_id`. A migração para `list` foi local e manteve a suíte verde: todo acesso por chave já passava por um único ponto (`get_session`), então a mudança se resumiu à declaração da estrutura, à troca de atribuição por `append` e aos dois métodos de listagem.

A escolha se justifica pelo acesso dominante: a coleção de sessões é **percorrida** (relatório, estatísticas, ordenação pelo critério que o usuário escolher) muito mais do que acessada por chave. E a ordem de chegada é informação de negócio — é ela que determina o `numero` sequencial. Lista preserva ordem; dicionário indexado por UUID não expressa ordem nenhuma.

Os três verbos que o enunciado cita explicitamente:

Verbo	Onde
<code>sessoes = []</code>	<code>session_manager.py:79</code>
<code>sessoes.append(...)</code>	<code>session_manager.py:172</code> (nova recarga) e <code>session_manager.py:181</code> (registro histórico)
<code>len(sessoes)</code>	<code>session_manager.py:483</code> e <code>menu.py:386</code>

2.4 Por que `_chargers` continua sendo um dicionário

Um corretor que abrir `session_manager.py` vai encontrar um dicionário convivendo com a lista. Isso é deliberado e não é meia-adoção da lista:

`session_manager.py`, linhas 56–86 — as duas estruturas e o critério de escolha de cada uma

```
class SessionManager:
    """
    Gerenciador de sessões de recarga simultâneas.
```

Suporta múltiplos postos (stations) e múltiplos carregadores por posto.
O estado é mantido em duas estruturas, cada uma escolhida pelo que a disciplina de Estruturas de Dados chamaria de acesso dominante:

```
_sessions : List[ChargingSession] – todas as sessões, na ordem em que
    entraram. É uma lista porque a coleção de sessões é percorrida
    (relatório, estatísticas, ordenação por critério escolhido pelo
    usuário) muito mais do que acessada por chave, e porque a ordem de
    chegada é informação de negócio: é ela que dá o ID sequencial.
    A consulta por ID usa `algoritmos.busca_sequencial`.

_chargers : {charger_id → session_id | None} – ocupação dos conectores.
    Continua dicionário de propósito: não é a coleção de sessões,
    é uma tabela fixa de 15 conectores físicos (3 postos × 5), com
    chave conhecida e imutável, escrita e lida a cada polling. Trocar
    por lista só acrescentaria varredura sem ganho nenhum.
"""

def __init__(self) -> None:
    self._sessions: List[ChargingSession] = []
    # Carregadores pré-cadastrados: 5 por posto, 3 postos = 15 total
    self._chargers: Dict[str, Optional[str]] = {
        f"P{p}-C{c}": None
        for p in range(1, 4)
        for c in range(1, MAX_CHARGERS_PER_STATION + 1)
    }
    logger.info("SessionManager inicializado com %d carregadores.", len(self._chargers))
```

`_chargers` **não é a coleção de sessões**. É a tabela dos conectores físicos — 3 postos × `MAX_CHARGERS_PER_STATION` (`session_manager.py:48`, valor 5) = 15 entradas, com chaves conhecidas e imutáveis (`P1-C1 ... P3-C5`), lida e escrita a cada ciclo de *polling*. Acesso por chave é o acesso dominante ali, e trocar por lista só acrescentaria varredura sem ganho nenhum. O enunciado pede lista para **armazenar as sessões**, e é isso que foi feito.

2.5 Dois métodos para dois fatos diferentes

O sistema já tinha `create_session`, que **inicia uma recarga agora**: valida o conector, recusa se estiver ocupado e marca a ocupação. Registrar um fato passado é outra coisa, e ganhou método próprio nesta Sprint:

`session_manager.py`, linhas 181–211

```
def registrar_historico(self, sessao: ChargingSession) -> ChargingSession:
    """
    Anexa uma sessão JÁ ENCERRADA à coleção, sem tocar na ocupação de
    conectores.

    Por que não reusar `create_session`: aquele método inicia uma recarga
    AGORA – valida o conector, recusa se estiver ocupado e marca a
    ocupação. São 15 conectores no sistema; carregar algumas centenas de
    registros históricos por ali quebraria no 16º, e registrar um fato
    passado não deve reservar hardware no presente. São duas operações
```

distintas e cada uma tem seu método.

É o caminho usado por ``db.carregar_sessoes()`` (histórico do SQLite) e pela opção "Nova sessão de recarga" do menu de terminal.

Args:

`sessao` : sessão pronta, com ``numero`` já atribuído

Returns:

A própria sessão, agora na coleção.

Raises:

`ValueError` : se ``numero`` já existir na coleção (item 8 do enunciado – não permitir ID duplicado). A checagem usa ``algoritmos.numero_existe``, ou seja, a busca sequencial avaliada na Sprint.

"""

```
if algoritmos.numero_existe(self._sessions, sessao.numero):
    raise ValueError(f"Já existe sessão com ID {sessao.numero}.")
self._sessions.append(sessao)
return sessao
```

Sem essa separação, carregar o histórico do banco quebraria no 16º registro (são 15 conectores) e o cadastro pelo menu tentaria reservar hardware no presente para descrever o passado. É também esse método que dá conteúdo real à validação de ID duplicado exigida pelo item 8: com `numero` apenas automático, "impedir ID duplicado" seria uma verificação vazia.

3. Funcionamento geral

3.1 De onde vêm os dados

`db.py`, linhas 217–247 — a ponte entre o SQLite e a coleção do menu

```
def carregar_sessoes() -> List[ChargingSession]:
    """
    Reconstrói as sessões pagas do histórico como objetos `ChargingSession`.

    Ordena por data de início e numera sequencialmente a partir de 1: esse
    `numero` é o ID que o enunciado da Sprint usa, o que o menu aceita
    digitado e a chave da busca binária. Ordenar por `inicio` no SQL é
    deliberado e não conflita com o conteúdo avaliado – quem ordena aqui é o
    banco de dados, com a cláusula ORDER BY, e o que está sendo estabelecido é
    a numeração dos registros, não o algoritmo de ordenação da Sprint. As
    ordenações que o usuário pede no menu passam por `algoritmos.bubble_sort`
    e `algoritmos.insertion_sort`.

    **NUNCA escreve no banco.** O menu é ferramenta de leitura e análise; a
    sessão cadastrada à mão nele vive só na memória daquela execução.
```

Returns:

Lista de sessões, possivelmente vazia. Banco ausente, tabela ainda não criada ou arquivo corrompido devolvem ****lista vazia**** em vez de exceção: o item 8 do enunciado exige que o programa não encerre inesperadamente, e faturamento não pode derrubar o processo por causa de leitura. O menu abre vazio e o cadastro manual preenche.

"""

try:

```
linhas = query_all(
    "SELECT * FROM sessoes WHERE metodo_pagto <> '"
    " ORDER BY inicio, session_id"
)
```

except sqlite3.Error as erro:

```
logger.warning("Histórico indisponível (%s) – carregando vazio.", erro)
return []
```

O banco atual traz **180 sessões pagas**, geradas pelas regras reais do produto (`seed_historico.py`): horário sorteado de uma distribuição com pico às 19h, tarifa calculada pela `PricingEngine` de verdade, potência entre o piso trifásico de 4,2 kW e o nominal de 11 kW do GW11K-HCA-20. O menu abre com dados com a mesma estrutura estatística da operação simulada, não com números inventados.

Banco ausente, tabela inexistente ou arquivo corrompido devolvem **lista vazia** em vez de exceção. O menu abre vazio e o cadastro manual preenche — exigência do item 8 e também das restrições do projeto, porque um sistema de faturamento não pode cair por causa de uma leitura.

3.2 O menu

`menu.py`, linhas 71–86

```
def exibir_menu() -> None:
    """Mostra as opções disponíveis (formato do enunciado, com uma a mais)."""
    print()
    print("=" * 37)
    print("      ESTAÇÃO DE RECARGA".ljust(37))
    print("      ChargeGrid Intelligence".ljust(37))
    print("=" * 37)
    print()
    print("1 - Nova sessão de recarga")
    print("2 - Listar sessões")
    print("3 - Buscar sessão")
    print("4 - Ordenar sessões")
    print("5 - Estatísticas")
    print("6 - Comparar algoritmos (Big-O na prática)")
    print("7 - Encerrar")
    print()
```

O enunciado pede um menu "semelhante a" e permite acrescentar opções. A opção 6 foi acrescentada porque é ela que produz as medições da §7: sem ela, os números da análise de complexidade seriam afirmações sem evidência.

3.3 Divisão em funções

Módulo	Responsabilidade	Funções
<code>algoritmos.py</code>	algoritmos puros sobre a coleção	<code>_por_numero</code> , <code>busca_sequencial</code> , <code>busca_binaria</code> , <code>bubble_sort</code> , <code>insertion_sort</code> , <code>estatisticas</code> , <code>numero_existe</code> , <code>proximo_numero</code>
<code>menu.py</code>	interação de terminal	<code>titulo</code> , <code>aviso</code> , <code>exibir_menu</code> , <code>ler_texto</code> , <code>ler_numero</code> , <code>ler_opcao</code> , <code>detalhar</code> , <code>imprimir_tabela</code> , <code>cadastrar_sessao</code> , <code>listar_sesoes</code> , <code>buscar_sessao</code> , <code>ordenar_sesoes</code> , <code>mostrar_estatisticas</code> , <code>_amostras</code> , <code>comparar_algoritmos</code> , <code>carregar_colecao</code> , <code>main</code> , <code>autoteste</code>
<code>session_manager.py</code>	ciclo de vida da sessão	<code>create_session</code> , <code>registrar_historico</code> , <code>get_session</code> , <code>list_active</code> , <code>list_all</code> , <code>finish_session</code> , ...
<code>power_manager.py</code>	distribuição de potência	<code>allocate</code> , <code>rebalance</code> , <code>throttle</code> , ...
<code>db.py</code>	persistência SQLite	<code>carregar_sesoes</code> , <code>query_all</code> , <code>execute</code> , ...

`algoritmos.py` importa **apenas** `models` e a biblioteca padrão. Não conhece Flask, não conhece SQLite, não importa `session_manager`. Esse isolamento tem três efeitos: o módulo avaliado pode ser lido sozinho, o `menu.py` roda sem dependência externa, e o teste-guarda (§8) pode varrer **só esse arquivo** procurando recursos nativos de ordenação.

3.4 Validações (item 8)

Situação	Tratamento	Onde
Texto onde se espera número	<code>try/except ValueError</code> , até 3 tentativas, mensagem dizendo quantas restam	<code>menu.py:124</code>
Valor negativo ou fora de faixa	recusado com a faixa na mensagem (<code>energia ≥ 0</code> , <code>tempo ≥ 0,1 min</code> , <code>custo ≥ 0</code>)	<code>menu.py:124</code>
ID duplicado	<code>numero_existe</code> (busca sequencial) antes de inserir; <code>registrar_historico</code> levanta <code>ValueError</code> como segunda barreira	<code>algoritmos.py:327</code> , <code>session_manager.py:181</code>
Opção de menu inválida	avisa e reexibe o menu, sem sair do laço	<code>menu.py:669</code>
Conector em formato inválido	recusado antes de construir a sessão	<code>menu.py:316</code>
<code>Ctrl+C</code> / <code>Ctrl+D</code>	<code>except (EOFError, KeyboardInterrupt)</code> encerra sem <i>traceback</i>	<code>menu.py:669</code>
Falha inesperada em qualquer opção	<code>except Exception</code> no laço: avisa, mantém a coleção e reexibe o menu	<code>menu.py:669</code>
Coleção vazia nas estatísticas	devolve zeros (as funções nativas de mínimo e máximo levantariam <code>ValueError</code>)	<code>algoritmos.py:249</code>
Banco ausente ou corrompido	lista vazia, com aviso no log	<code>db.py:217</code>

O usuário também pode desistir de qualquer campo com Enter vazio, o que evita que uma digitação errada no primeiro campo obrigue a completar o cadastro inteiro.

4. Algoritmo de busca

Duas buscas foram implementadas, sem nenhuma função pronta de procura.

4.1 Busca sequencial — usada em produção

`algoritmos.py`, linhas 69–99

```
def busca_sequencial(colecao: List[ChargingSession],
                    alvo: Any,
                    chave: Chave = _por_numero) -> Tuple[int, int]:
    """
    Busca sequencial (linear) – percorre a coleção do início ao fim.

    Args:
        colecao : lista de sessões, em qualquer ordem
        alvo    : valor procurado
        chave   : extrator do valor a comparar (padrão: `numero`)

    Returns:
        (índice do elemento, comparações realizadas) – índice -1 se ausente.

    Complexidade:  $O(n)$ .

    De onde vem o crescimento: do laço `for indice in range(len(colecao))`,
    que no pior caso (elemento ausente, ou último da lista) executa uma
    comparação por elemento. Dobrar o número de sessões dobra o trabalho.
    Melhor caso  $O(1)$ , quando o alvo é o primeiro elemento.

    Não exige pré-condição nenhuma – é o que a torna a busca usada em
    produção pelo `SessionManager.get_session`, onde a lista de sessões está
    em ordem de criação e não em ordem de ID.
    """
    comparacoes = 0
    for indice in range(len(colecao)):
        comparacoes += 1
        if chave(colecao[indice]) == alvo:
            return indice, comparacoes
    return -1, comparacoes
```

Onde o sistema usa. É a busca de `SessionManager.get_session`, ou seja, a que responde toda consulta de sessão da camada web:

`session_manager.py`, linhas 438–454

```
def get_session(self, session_id: str) -> Optional[ChargingSession]:
    """
    Retorna a sessão ou None se não encontrada.

    Usa busca sequencial (algoritmos.busca_sequencial) sobre a lista,
    com session_id como chave. É o mesmo algoritmo avaliado na Sprint,
```

exercitado em produção: toda consulta da camada web passa por aqui.

Complexidade: $O(n)$. O n relevante é pequeno por construção física – são 15 conectores, logo no máximo 15 sessões ativas simultâneas; as encerradas e pagas saem da memória para o SQLite. Nenhuma consulta da camada web percorre o histórico inteiro.

```
"""
indice, _ = algoritmos.busca_sequencial(
    self._sessions, session_id, chave=lambda s: s.session_id
)
return self._sessions[indice] if indice >= 0 else None
```

O extrator de chave (`chave`) é o que permite a mesma função servir aos dois casos: o menu procura por `numero` (inteiro digitado), a camada web procura por `session_id` (UUID). Uma função, dois usos — em vez de duas implementações do mesmo laço.

4.2 Busca binária — o contraste

`algoritmos.py`, linhas 102–144

```
def busca_binaria(colecao: List[ChargingSession],
                  alvo: Any,
                  chave: Chave = _por_numero) -> Tuple[int, int]:
    """
    Busca binária – divide o intervalo de procura pela metade a cada passo.

    PRÉ-CONDIÇÃO: `colecao` ordenada de forma crescente pela mesma `chave`
    usada aqui. Sobre lista desordenada o resultado é indefinido – quem chama
    é responsável por ordenar antes (o menu ordena com `insertion_sort` e
    avisa isso na tela).

    Args:
        colecao : lista de sessões ORDENADA pela chave
        alvo    : valor procurado
        chave   : extrator do valor a comparar (padrão: `numero`)

    Returns:
        (índice do elemento, comparações realizadas) – índice -1 se ausente.

    Complexidade:  $O(\log n)$ .

    De onde vem o crescimento: do laço `while inicio <= fim`, cujo intervalo
    de procura (`fim - inicio`) é dividido por 2 em cada iteração. Para chegar
    de  $n$  a 1 dividindo por 2 são necessários  $\log_2(n)$  passos – com 180 sessões,
    no máximo 8 comparações contra as 180 da busca sequencial. É o contraste
    que o item 10 do enunciado pede.
    """
    comparacoes = 0
    inicio = 0
    fim = len(colecao) - 1
```

```

while inicio <= fim:
    meio = (inicio + fim) // 2
    comparacoes += 1
    valor = chave(colecao[meio])
    if valor == alvo:
        return meio, comparacoes
    if valor < alvo:
        inicio = meio + 1
    else:
        fim = meio - 1

return -1, comparacoes

```

Onde o sistema usa. Na opção 3 do menu, quando o operador a escolhe. A pré-condição (lista ordenada) não é escondida: o menu ordena com `insertion_sort`, informa na tela quantas comparações a ordenação custou e só então busca, deixando visível que o custo real da binária sobre uma lista desordenada é **ordenar + buscar**.

`menu.py`, linhas 434–448 — a pré-condição, explicitada na tela

```

print(" Pré-condição da busca binária: lista ordenada por ID.")
inicio = time.perf_counter()
comp_ordenacao = algoritmos.insertion_sort(colecao,
                                            algoritmos.CRITERIOS["1"][1])

t_ordenacao = (time.perf_counter() - inicio) * 1000
print(f" Ordenando com insertion sort: {comp_ordenacao} "
      f"comparação(ões), {t_ordenacao:.3f} ms")

inicio = time.perf_counter()
indice, comparacoes = algoritmos.busca_binaria(colecao, alvo)
t_busca = (time.perf_counter() - inicio) * 1000
print(f" Busca binária em {len(colecao)} sessões: "
      f"{comparacoes} comparação(ões), {t_busca:.3f} ms")
print(f" Custo total (ordenar + buscar): "
      f"{comp_ordenacao + comparacoes} comparação(ões)")

```

5. Algoritmo de ordenação

Duas ordenações foram implementadas, sem usar a ordenação nativa de listas. O usuário escolhe o critério entre quatro:

`algoritmos.py`, linhas 237–242 — os critérios oferecidos ao usuário

```

CRITERIOS: Dict[str, Tuple[str, Chave]] = {
    "1": ("ID", lambda s: s.numero),
    "2": ("Energia consumida", lambda s: s.energy_kwh),
    "3": ("Custo da sessão", lambda s: s.total_cost_brl),
    "4": ("Tempo de recarga", lambda s: s.duration_minutes),
}

```

5.1 Bubble sort

`algoritmos.py`, linhas 151–185

```
def bubble_sort(colecao: List[ChargingSession], chave: Chave) -> int:
    """
    Bubble sort – compara pares vizinhos e troca os que estão fora de ordem,
    repetindo até que o maior elemento tenha "borbulhado" para o fim.

    Ordena in-place (a lista recebida é modificada) e é estável:
    sessões com a mesma chave preservam a ordem relativa original.

    Args:
        colecao : lista a ordenar, modificada no lugar
        chave   : extrator do valor de comparação

    Returns:
        Comparações realizadas – exatamente  $n(n-1)/2$ .

    Complexidade:  $O(n^2)$  em todos os casos.

    De onde vem o crescimento: dos dois laços `for` aninhados. O externo roda
    n-1 vezes; o interno roda n-1-i vezes. A soma é  $n(n-1)/2$  comparações, um
    polinômio de grau 2 – daí o  $n^2$ .

    Decisão deliberada: sem parada antecipada. A otimização clássica
    (interromper quando uma passada não troca nada) derrubaria o melhor caso
    para  $O(n)$ , mas quebraria a igualdade exata com  $n(n-1)/2$  que a tabela do
    relatório usa para provar a fórmula. Quem precisa de melhor caso linear
    usa insertion_sort, que oferece isso por construção.
    """
    comparacoes = 0
    n = len(colecao)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            comparacoes += 1
            if chave(colecao[j]) > chave(colecao[j + 1]):
                colecao[j], colecao[j + 1] = colecao[j + 1], colecao[j]
    return comparacoes
```

Onde o sistema usa. Na opção 4 do menu, quando o operador a escolhe.

Decisão registrada: o bubble sort foi deixado **sem parada antecipada**. A otimização clássica (interromper quando uma passada não troca nada) daria a ele um melhor caso $O(n)$, mas quebraria a igualdade exata com $n(n-1)/2$ que a tabela da §7 usa para provar a fórmula. Quem precisa de melhor caso linear usa o insertion sort, que oferece isso por construção. Um teste automatizado (`test_bubble_sort_nao_tem_parada_antecipada`) garante que essa decisão não seja "otimizada" por acidente depois.

5.2 Insertion sort — usado em produção

`algoritmos.py`, linhas 188–230

```
def insertion_sort(colecao: List[ChargingSession], chave: Chave) -> int:
    """
    Insertion sort – percorre a lista da esquerda para a direita e insere cada
    elemento na posição correta dentro da parte já ordenada, deslocando os
    maiores uma casa para a direita.

    Ordena in-place e é estável (a parada usa <=, então iguais não
    se cruzam). A estabilidade importa no PowerManager.rebalance: sessões
    com a mesma potência alocada são restauradas na ordem em que entraram,
    o que torna a decisão reproduzível.

    Args:
        colecao : lista a ordenar, modificada no lugar
        chave   : extrator do valor de comparação

    Returns:
        Comparações realizadas – entre  $n-1$  e  $n(n-1)/2$ .

    Complexidade:  $O(n^2)$  no pior caso,  $O(n)$  no melhor.

    De onde vem o crescimento: do laço while j >= 0 aninhado no for. Ele
    recua enquanto encontra elementos maiores que o atual. Com a lista em
    ordem inversa, o i-ésimo elemento recua i posições e a soma volta a ser
     $n(n-1)/2$ . Com a lista já ordenada, o while para na primeira comparação
    e o total cai para  $n-1$  – linear.

    É esse melhor caso que o justifica no rebalance: a cada liberação de
    conector a lista de sessões em throttle chega quase ordenada, e são no
    máximo 5 elementos (MAX_CHARGERS_PER_STATION), ou seja  $\leq 10$  comparações.
    """
    comparacoes = 0
    for i in range(1, len(colecao)):
        atual = colecao[i]
        valor = chave(atual)
        j = i - 1
        while j >= 0:
            comparacoes += 1
            if chave(colecao[j]) <= valor:
                break
            colecao[j + 1] = colecao[j]
            j -= 1
        colecao[j + 1] = atual
    return comparacoes
```

Onde o sistema usa. Em `PowerManager.rebalance`, que roda toda vez que um conector é liberado e a potência precisa ser redistribuída entre as sessões que estavam em *throttle*:

```

# Ordena throttled: mais reduzidas primeiro (maior ganho por
# restauração). A ordenação é feita por `algoritmos.insertion_sort`,
# implementado à mão na Sprint 3 – não pela ordenação nativa da lista.
#
# Por que insertion sort aqui: a coleção é minúscula e quase sempre
# já quase ordenada. São no máximo MAX_CHARGERS_PER_STATION = 5
# sessões por posto, logo no máximo  $5 \cdot 4 / 2 = 10$  comparações no pior
# caso e apenas 4 no melhor (lista já ordenada), porque o laço interno
# para na primeira comparação. Um algoritmo  $O(n \log n)$  não teria onde
# ganhar com  $n \leq 5$ , e a estabilidade do insertion garante que sessões
# com a mesma potência sejam restauradas na ordem de chegada – a
# decisão fica reproduzível.
comparacoes = algoritmos.insertion_sort(
    throttled, lambda s: s.allocated_power_kw
)

```

O número de comparações vai para a mensagem do `AllocationResult` e aparece na tela do painel. Exemplo real, capturado do `/dashboard` com quatro sessões num posto:

```

✅ Rebalanceamento: 2 sessão(ões) restauradas (carga: 20.2/22.0 kW – 91.6%)
[insertion sort: 3 em fila, comparações: 2]

```

Por que insertion e não outro. A fila de restauração tem no máximo `MAX_CHARGERS_PER_STATION` = 5 elementos (`session_manager.py:48`), ou seja no máximo 10 comparações no pior caso e 4 no melhor. Com $n \leq 5$ um algoritmo $O(n \log n)$ não tem onde ganhar, e a **estabilidade** do insertion garante que sessões com a mesma potência alocada sejam restauradas na ordem de chegada — a decisão de potência fica reproduzível, o que importa num sistema que audita faturamento.

6. Análise de complexidade (Big-O)

O enunciado exige a análise de **dois** algoritmos do próprio projeto. Os quatro são analisados aqui, começando pelos dois que o enunciado dá como exemplo. Para cada um está apontado o **trecho que provoca o crescimento** — exigência literal do item 9.

6.1 Visão geral

Algoritmo	Melhor	Pior	Trecho que provoca o crescimento	Onde o sistema usa
<code>busca_sequencial</code>	$O(1)$	$O(n)$	<code>for indice in range(len(colecao))</code> — <code>algoritmos.py:95</code>	<code>SessionManager.get_session</code> (<code>session_manager.py:438</code>)
<code>bubble_sort</code>	$O(n^2)$	$O(n^2)$	os dois <code>for</code> aninhados — <code>algoritmos.py:180–181</code>	opção 4 do menu
<code>busca_binaria</code>	$O(1)$	$O(\log n)$	<code>while inicio <= fim</code> com <code>meio = (inicio + fim) // 2</code> — <code>algoritmos.py:133</code>	opção 3 do menu
<code>insertion_sort</code>	$O(n)$	$O(n^2)$	o <code>while j >= 0</code> aninhado no <code>for</code> — <code>algoritmos.py:223</code>	<code>PowerManager.rebalance</code> (<code>power_manager.py:396</code>)

6.2 busca_sequencial — $O(n)$

O laço percorre a lista posição por posição e faz **uma comparação por elemento**. O trecho responsável é o `for` que varre `range(len(colecao))`: o número de iterações é exatamente o tamanho da entrada.

- **Melhor caso, $O(1)$:** o alvo é o primeiro elemento — 1 comparação.
- **Pior caso, $O(n)$:** o alvo é o último, ou não existe — n comparações.
- **Caso médio:** $(n+1)/2$ comparações para um alvo presente em posição uniforme.

Medido em produção, com cinco sessões ativas no `SessionManager` (`get_session` procurando por `session_id`):

Posição da sessão na lista	Comparações
1	1
2	2
3	3
4	4
5	5

A relação é linear e exata: posição k custa k comparações.

Por que $O(n)$ é aceitável aqui. O n relevante para o `get_session` é o número de sessões **em memória**, e esse número é limitado pela física da instalação: 15 conectores, logo no máximo 15 sessões ativas simultâneas. As sessões encerradas e pagas são arquivadas no SQLite e saem da lista. Nenhuma consulta da camada web percorre o histórico inteiro. Trocar por uma estrutura de acesso $O(1)$ resolveria um problema que o sistema não tem.

6.3 bubble_sort — $O(n^2)$

O laço externo executa $n-1$ vezes; o interno executa $n-1-i$ vezes. A soma é

$$(n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2$$

um polinômio de grau 2 — daí o n^2 . **O trecho responsável é o aninhamento dos dois `for`:** cada elemento do laço externo obriga uma nova varredura no interno.

Como não há parada antecipada, o número de comparações é **igual em todos os casos** e igual à fórmula. A tabela da §7.2 confirma isso para todos os tamanhos medidos.

6.4 busca_binaria — $O(\log n)$

A cada iteração o intervalo de procura (`fim - inicio`) é **dividido por dois**. Para reduzir n a 1 dividindo por 2 são necessários $\lceil \log_2(n) \rceil$ passos, e cada iteração faz uma comparação. **O trecho responsável é o `while` combinado com `meio = (inicio + fim) // 2`:** é a divisão do intervalo que faz o custo crescer logaritmicamente em vez de linearmente.

- **Melhor caso, $O(1)$:** o alvo está exatamente no meio.
- **Pior caso, $O(\log n)$:** o alvo está numa ponta ou não existe.
- **Pré-condição:** lista ordenada pela mesma chave. Sem isso o resultado é indefinido — não é uma otimização, é um requisito de correteude.

Com 180 sessões, $\lceil \log_2(180) \rceil = 8$ comparações no pior caso, contra 180 da sequencial.

6.5 `insertion_sort` — $O(n^2)$ pior, $O(n)$ melhor

O `for` externo percorre os $n-1$ elementos a inserir; o `while` interno recua enquanto encontra elementos maiores que o atual. O trecho responsável é esse `while` aninhado. O que o distingue do bubble é que o `while` para na primeira comparação falsa:

- **Melhor caso, $O(n)$:** lista já ordenada — 1 comparação por elemento, total $n-1$.
- **Pior caso, $O(n^2)$:** lista em ordem inversa — o i -ésimo elemento recua i posições, e a soma volta a ser $n(n-1)/2$.
- **Caso médio:** $\approx n^2/4$, metade do bubble, porque em média cada elemento recua metade da parte já ordenada.

É essa sensibilidade à ordem prévia que o justifica no `rebalance`: a fila de sessões em *throttle* chega quase ordenada (a potência alocada muda pouco entre ciclos), então o custo real fica perto do melhor caso.

7. Comparação e significado prático

Todos os números desta seção foram **medidos** pela opção 6 do menu (`menu.py:552`) sobre a coleção real de 180 sessões, nesta máquina. Tempos são o menor de 5 execuções, em milissegundos. A §7.4 é projeção e está marcada como tal.

7.1 Busca: sequencial × binária

Alvo ausente da coleção — pior caso para as duas.

n (sessões)	sequencial $O(n)$	binária $O(\log n)$	redução	tempo seq.	tempo bin.
10	10	4	60.0%	0.0010 ms	0.0010 ms
25	25	5	80.0%	0.0019 ms	0.0008 ms
50	50	6	88.0%	0.0033 ms	0.0009 ms
100	100	7	93.0%	0.0063 ms	0.0010 ms
180	180	8	95.6%	0.0110 ms	0.0011 ms

Leitura. Multiplicar n por 18 (de 10 para 180) multiplica o trabalho da sequencial por 18 e **soma 4 comparações** na binária. É a diferença entre crescer proporcionalmente à entrada e crescer proporcionalmente ao número de vezes que a entrada pode ser dividida por 2.

O que isso significa para o produto. Nada, na escala atual: 180 comparações em 0.0110 ms é irrelevante. A diferença passa a importar quando a mesma operação entra num laço quente — por exemplo, o *polling* de status que a interface web faz a cada 5 segundos para cada sessão ativa. É por isso que a escolha do `get_session` foi documentada em vez de apenas feita: a justificativa (n limitado a 15 por construção física) é o que torna $O(n)$ defensável ali, e é ela que deixa de valer se o sistema passar a manter o histórico em memória.

7.2 Ordenação: bubble × insertion

Ordenando por energia consumida, a partir da ordem original do histórico.

n	bubble	$n(n-1)/2$	insertion	insertion já ordenada	$n-1$	tempo bubble	tempo insertion
10	45	45	32	9	9	0.007 ms	0.004 ms
25	300	300	193	24	24	0.042 ms	0.019 ms
50	1.225	1.225	678	49	49	0.166 ms	0.066 ms
100	4.950	4.950	2.387	99	99	0.648 ms	0.231 ms
180	16.110	16.110	7.933	179	179	2.160 ms	0.778 ms

Três leituras.

1. A coluna `bubble` é **idêntica** à coluna `n(n-1)/2` em todas as linhas. A fórmula fechada da análise não é uma estimativa: é o número exato de comparações que o código executa.
2. A coluna `insertion` fica em torno de metade da `bubble` — é o caso médio $\approx n^2/4$ contra $n^2/2$. As duas são $O(n^2)$: dobrar n quadruplica as duas colunas. Com n de 10 para 180 (fator 18), as comparações do bubble vão de 45 para 16.110 — fator 358, contra $18^2 = 324$ previstos pelo termo n^2 . A diferença vem do termo $-n$ de $n(n-1)/2$, que pesa proporcionalmente mais quando n é pequeno: com $n = 10$ a fórmula ainda está 10% abaixo de $n^2/2$.
3. A coluna `insertion já ordenada` é **exatamente $n-1$** em todas as linhas. Aí o insertion deixa de ser quadrático e passa a ser linear. É a única diferença assintótica entre os dois algoritmos, e é ela que decide qual usar no `rebalance`.

7.3 O caso real do `rebalance`

	Valor
n máximo (conectores por posto)	5
Comparações no pior caso	10
Comparações no melhor caso	4
Medido no <code>/dashboard</code> com 3 em fila	2

Com $n \leq 5$, a distância entre $O(n^2)$ e $O(n \log n)$ é de unidades de comparação. A decisão correta de engenharia aqui não é escolher o algoritmo assintoticamente melhor, é escolher o mais simples que atende — e registrar por escrito qual é o limite que torna isso verdade. Se `MAX_CHARGERS_PER_STATION` virasse 500, a conclusão mudaria.

7.4 Projeção para n grande

Estes números são projeção pela fórmula, não medição. Servem para mostrar onde cada classe de complexidade deixa de ser viável.

n	sequencial $O(n)$	binária $O(\log n)$	bubble / insertion pior $O(n^2)$
1.000	1.000	10	499.500
10.000	10.000	14	49.995.000
100.000	100.000	17	4.999.950.000
1.000.000	1.000.000	20	499.999.500.000

Com 1 milhão de sessões — um ano de operação de uma rede média — a busca binária resolve em 20 comparações o que a sequencial resolve em 1 milhão, e as ordenações quadráticas pedem meio trilhão de comparações. É nesse ponto que a resposta deixa de ser "escolha o algoritmo certo" e passa a ser "não carregue tudo em memória": paginação e índice no banco, que é justamente o caminho do Sprint 4.

8. Sobre os recursos nativos que permanecem no sistema

O enunciado autoriza explicitamente:

Recursos nativos do Python poderão ser utilizados nas demais partes do sistema quando não substituírem diretamente o conteúdo algorítmico solicitado.

Esta seção lista **todas** as ocorrências que sobraram, para que ninguém precise deduzir se alguma delas substitui algoritmo avaliado. Nenhuma substitui.

Ocorrência	Onde	Para que serve	Substitui algoritmo avaliado?
<code>historico_consolidado.sort(key=...)</code>	<code>app.py:844</code>	ordenar as últimas decisões de potência dos 3 postos para exibição no painel	Não. É apresentação de log, não a coleção de sessões
<code>sorted(_posto_sms[pid].list_active(), key=...)</code>	<code>app.py:850</code>	agrupar as sessões de um posto por conector na tela	Não. Ordena por conector para layout, não por critério de negócio escolhido pelo usuário
<code>ORDER BY inicio, session_id</code>	<code>db.py:243</code>	estabelecer a numeração sequencial ao carregar o histórico	Não. Quem ordena é o SQLite, e o resultado é a numeração dos registros, não a ordenação pedida no item 5
<code>_chargers: Dict[str, Optional[str]]</code> e acessos por chave	<code>session_manager.py:81</code>	tabela fixa de 15 conectores físicos	Não. Não é a coleção de sessões (ver §2.4)
<code>dict</code> de despacho <code>ACOES</code> , <code>CRITERIOS</code>	<code>menu.py</code> , <code>algoritmos.py:237</code>	mapear tecla → função / tecla → critério	Não. É despacho de menu
<code>max()</code> , <code>min()</code> , <code>sum()</code>	não usados em <code>algoritmos.py</code>	—	—

Dentro de `algoritmos.py` não há `.sort()`, `sorted()`, `.index()` nem `bisect`, e isso é verificado automaticamente pela suíte:

`test_chargegrid.py`, linhas 1898–1908

```
def test_algoritmos_nao_usam_sort_nem_sorted(self):
    """
    O enunciado permite recursos nativos "nas demais partes do sistema",
    mas não dentro dos algoritmos avaliados. Esta guarda impede que alguém
    "simplifique" algoritmos.py depois – seria reprovação direta.
    """
    import algoritmos

    fonte = pathlib.Path(algoritmos.__file__).read_text(encoding="utf-8")
    for proibido in (".sort(", "sorted(", ".index(", "bisect"):
        assert proibido not in fonte, f"{proibido} em algoritmos.py"
```

As estatísticas também não usam as funções nativas de agregação: mínimo, máximo e somas são acompanhados numa única varredura explícita (`algoritmos.py:249`).

9. Como conferir esta entrega

Comando	O que prova
<code>python -m pytest -q</code>	181 testes verdes, incluindo 51 específicos desta Sprint

Comando	O que prova
<code>python menu.py --autoteste</code>	26 assertões sobre algoritmos e integração, sem <code>pytest</code> instalado
<code>python menu.py</code>	o menu, com o histórico de 180 sessões carregado
<code>python app.py</code>	a interface web, onde o insertion sort aparece na mensagem de rebalanceamento

A suíte cobre, entre outras coisas: as contagens de comparações contra as fórmulas fechadas, a estabilidade das duas ordenações, a coleção vazia nas estatísticas, a recusa de ID duplicado, a resiliência do carregamento a banco corrompido, e dois testes-espião que falham se o `get_session` parar de usar a busca sequencial ou se o `rebalance` parar de usar o insertion sort.

Este documento também é verificado por script: cada bloco `python` acima é **fatiado do arquivo-fonte no momento da geração**, e cada âncora de linha é conferida contra o arquivo. Nenhum trecho de código foi digitado à mão neste relatório.

Dependências do projeto: `flask` e `pytest`. Nada mais. `sqlite3` é da biblioteca padrão, e `algoritmos.py` e `menu.py` rodam sem nem isso instalado.

10. Resumo das decisões

Decisão	Alternativa descartada	Por quê
Algoritmos dentro do app, rodando em produção	programa separado para a prova	o enunciado pede evolução do sistema e análise do "código efetivamente desenvolvido"
<code>list</code> para as sessões	<code>dict</code> indexado por UUID	o acesso dominante é varredura, e a ordem de chegada é informação de negócio
<code>dict</code> mantido para <code>_chargers</code>	converter tudo em lista	15 chaves fixas, acesso por chave dominante; converter só acrescentaria varredura
<code>numero</code> inteiro ao lado do <code>session_id</code>	usar o UUID como chave de busca	UUID não é digitável nem serve de chave ordenável para a binária
Duração derivada dos carimbos de tempo	campo <code>tempo</code> independente	energia é potência × tempo; dois valores de duração poderiam divergir
<code>registrar_historico</code> separado de <code>create_session</code>	reusar <code>create_session</code>	registrar o passado não deve reservar conector no presente
Bubble sort sem parada antecipada	versão otimizada	preserva a igualdade exata com $n(n-1)/2$ usada na análise
Insertion sort no <code>rebalance</code>	ordenação nativa, ou $O(n \log n)$	$n \leq 5$; melhor caso linear e estabilidade importam mais que a assíntota
<code>algoritmos.py</code> isolado	algoritmos dentro do <code>session_manager</code>	responsabilidade única, e permite ao teste-guarda varrer só o arquivo avaliado